Corso di Programmazione Web – Esercitazione – 22 Maggio 2026

Tutto il lavoro deve essere svolto individualmente.

Rispondere a ogni domanda in modo chiaro, utilizzando schizzi, tabelle o descrizioni dove necessario. Non scrivere codice, ma fornire dettagliate progettazioni API. Scrivere nome, cognome, numero di matricola nel foglio dove si svolge l'esame. Per risparmiare spazio e tempo, si può usare il seguente formato per descrivere le API:

```
{ idEsempio : int : 1             // assegnato dal server
  campoEsempio : str : "esempio"  // obbligatorio
  altroCampo : int : 1            // opzionale, minimo = 0, massimo = 10 }
```

Sistema di Gestione Turni di Volontariato

Progettare un'applicazione web che consenta agli utenti di organizzare e partecipare ad attività di volontariato nella propria città (es. pulizia parchi, raccolta fondi, supporto anziani, ecc.).

API richieste

- API 1: Inserire una nuova attività di volontariato
- API 2: Consultare la lista di attività disponibili
- API 3: Consultare il dettaglio di una delle attività disponibili

Requisiti dell'applicazione

Ogni attività ha: idAttivita, titolo, descrizione, luogo, data, oraInizio, oraFine, categoria, idOrganizzatore.

Gli utenti sono identificati da un idUtente già esistente nel server (non serve API di registrazione).

Specifiche tecniche

- idAttivita viene assegnato dal server.
- Si assuma che l'utente organizzatore sia assegnato dal server (l'utente ha effettuato il login).
- oraInizio e oraFine sono nel formato HH:MM; oraFine deve essere successiva a oraInizio.
- Data è nel formato DD-MM-YYYY.
- Le categorie sono limitate ai valori: "ambiente", "sociale", "educazione", "salute", "altro".

Domanda 1 – Progettazione delle API (15 punti)

Progettare le tre API descritte sopra. Per ciascuna API:

- 1 punto: endpoint e metodo HTTP
- 2 punti: modello di richiesta/risposta completo con tipi di dato e vincoli (inclusi quali campi sono obbligatori/opzionali, quali sono assegnati dal server, ed eventuali massimi e minimi)
- 1 punto: esempio concreto (richiesta e risposta)

Infine, 3 punti in totale sono assegnati in base alla chiarezza e completezza delle informazioni che complementano il servizio (per esempio, menzione specifica di controlli logici/validazioni server-side).

Domanda 2 – Validazione e gestione errori (5 punti)

- 1 punto: descrizione dei seguenti codici di errore (404, 422, 500). Elaborare la risposta – non è sufficiente scrivere il "titolo" del codice di errore.
- 2 punti per ogni esempio: fornire 2 esempi concreti (uno per ogni errore) con messaggio JSON e spiegazione, di richieste che generano i codici di errore 404 e 422. Gli errori devono riferirsi chiaramente alle API appena progettate.

Domanda 3 – Estensione (3 punti)

Attualmente, il server permette a chiunque di inserire una nuova attività. Descrivere quali modifiche sono da applicare per fare in modo che solo gli utenti *amministratori* possano farlo.

MODELLO ATTIVITÀ ⟶ {

"idAttività" : INT : 1, // ASSEGNATO DAL SERVER, UNIVOCO

"Titolo" : STR : "Pulizia canale", // OBBLIGATORIO, MIN=1, MAX=50 CARATTERI

"Descrizione" : STR : "non lo so", // OBBLIGATORIO, MIN=1, MAX=100 CARATTERI

"Luogo" : STR : "Cagliari", // OBBLIGATORIO

"Data" : STR : "01-06-2026", // OBBLIGATORIO, FORMATO VALIDO GG-MM-YYYY

"OraInizio" : STR : "10:00", // OBBLIGATORIO, FORMATO VALIDO HH:MM

"OraFine" : STR: "12:00", // OBBLIGATORIO, FORMATO VALIDO HH:MM, ORAFINE DEVE ESSERE SUCCESSIVA A OraInizio

"Categoria" : STR : "Ambiente", //OBBLIGATORIO, VALORI AMMESSI : "Ambiente", "Sociale", "Educazione", "Salute", "Altro"

"idOrganizzatore" : INT : 5 // ASSEGNATO DAL SERVER DALL'UTENTE LOGGATO

}

API 1 - INSERIREMENTO NUOVA ATTIVITÀ DI VOLONTARIATO

POST /Attività ⟶

SI USA POST PERCHÈ L'UTENTE INVIA DATI AL SERVER PER CREARE UNA NUOVA ATTIVITÀ DI VOLONTARIATO

⟵ 201 CREATED

{

"Titolo" : STR : "Pulizia canale", //OBBLIGATORIO, MIN=1, MAX=50 CARATTERI

"Descrizione" : STR : "non lo so", //OBBLIGATORIO, MIN=1, MAX=100 CARATTERI

"Luogo" : STR : "Cagliari", //OBBLIGATORIO

"Data" : STR : "01-06-2026", //OBBLIGATORIO, FORMATO VALIDO GG-MM-YYYY

"OraInizio" : STR : "10:00", //OBBLIGATORIO, FORMATO VALIDO HH:MM

"OraFine" : STR: "12:00", // OBBLIGATORIO, FORMATO VALIDO HH:MM, ORAFINE DEVE ESSERE SUCCESSIVA A OraInizio

"Categoria" : STR : "Ambiente", //OBBLIGATORIO, VALORI AMMESSI : "Ambiente", "Sociale", "Educazione", "Salute", "Altro"

}

⟶ {

"idAttività" : INT : 1, // ASSEGNATO DAL SERVER

"Messaggio" : "Attività di volontariato inserito correttamente"

}

API 2 - CONSULTAZIONE LISTA ATTIVITÀ DISPONIBILI

GET /Attività ⟶

USIAMO GET PERCHÈ VOGLIAMO LA LISTA DI ATTIVITÀ DISPONIBILI. NESSUN BODY

⟵ 200 OK

[

{

"idAttività" : INT : 1, // ASSEGNATO DAL SERVER

"Titolo" : STR : "Pulizia canale", //OBBLIGATORIO, MIN=1, MAX=50 CARATTERI

"Luogo" : STR : "Cagliari", //OBBLIGATORIO

"Data" : STR : "01-06-2026", //OBBLIGATORIO, FORMATO VALIDO GG-MM-YYYY

"OraInizio" : STR : "10:00", //OBBLIGATORIO, FORMATO VALIDO HH:MM

"OraFine" : STR: "12:00", // OBBLIGATORIO, FORMATO VALIDO HH:MM, ORAFINE DEVE ESSERE SUCCESSIVA A OraInizio

"Categoria" : STR : "Ambiente", //OBBLIGATORIO, VALORI AMMESSI : "Ambiente", "Sociale", "Educazione", "Salute", "Altro"

"idOrganizzatore" : INT : 5 //ASSEGNATO DAL SERVER

{ "altreAttività" : ... }

}

]

API 3 - CONSULTAZIONE DETTAGLIO DI UNA DELLE ATTIVITÀ DISPONIBILI

GET /Attività/{idAttività} ⟶

USIAMO GET PERCHÈ VOGLIAMO VEDERE IN DETTAGLIO UNA DELLE ATTIVITÀ DISPONIBILI. NESSUN BODY

⟵ 200 OK

{

"idAttività" : INT : 1, // ASSEGNATO DAL SERVER

"Titolo" : STR : "Pulizia canale", //OBBLIGATORIO, MIN=1, MAX=50 CARATTERI

"Descrizione" : STR : "non lo so", //OBBLIGATORIO, MIN=1, MAX=100 CARATTERI

"Luogo" : STR : "Cagliari", //OBBLIGATORIO

"Data" : STR : "01-06-2026", //OBBLIGATORIO, FORMATO VALIDO GG-MM-YYYY

"OraInizio" : STR : "10:00", //OBBLIGATORIO, FORMATO VALIDO HH:MM

"OraFine" : STR: "12:00", // OBBLIGATORIO, FORMATO VALIDO HH:MM, ORAFINE DEVE ESSERE SUCCESSIVA A OraInizio

"Categoria" : STR : "Ambiente", //OBBLIGATORIO, VALORI AMMESSI : "Ambiente", "Sociale", "Educazione", "Salute", "Altro"

"idOrganizzatore" : INT : 5 //ASSEGNATO DAL SERVER

}

- **404 NOT FOUND** ⟶ INDICA CHE LA RISORSA RICHIESTA NON ESISTE. SI USA QUANDO IL CLIENT CHIEDE UN'ATTIVITÀ O UN ENDPOINT INESISTENTE

POST /attività/100 →
← 404 NOT FOUND

{
"data": str: "04-06-2026", // OBBLIGATORIO, FORMATO VALIDO dd-mm-yyyy
"orainizio": str: "10:00", // OBBLIGATORIO, FORMATO VALIDO hh:mm
"orafine": str: "12:00", // OBBLIGATORIO, FORMATO VALIDO hh:mm, ORAFINE DEVE ESSERE SUCCESSIVA AD ORAINIZIO
"titolo": str: "Pulizia canale", //OBBLIGATORIO, MIN=1, MAX=50 CARATTERI
}

→
{
"error": "Attività con idattività 100 non esiste"
}

- **422 UNPROCESSABLE ENTITY** ⟶ INDICA CHE LA RICHIESTA È SINTATTICAMENTE CORRETTA, MA CONTIENE DATI CHE VIOLANO I VINCOLI LOGICI O DI VALIDAZIONE. SI USA QUANDO I DATI HANNO FORMATO ERRATO O NON RISPETTANO LE REGOLE DELLA TRACCIA: DATA NON NEL FORMATO dd-mm-yyyy, CATEGORIA NON AMMESSA, ORAFINE NON SUCCESSIVA AD ORAINIZIO

POST /attività →
← 422 UNPROCESSABLE ENTITY

{
"titolo": str: "Pulizia canale", //OBBLIGATORIO, MIN=1, MAX=50 CARATTERI
"descrizione": str: "non lo so", //OBBLIGATORIO, MIN=1, MAX=100 CARATTERI
"luogo": str: "Cagliari", //OBBLIGATORIO
"data": str: "04-06-2026", // OBBLIGATORIO, FORMATO VALIDO dd-mm-yyyy
"orainizio": str: "10:00", // OBBLIGATORIO, FORMATO VALIDO hh:mm
"orafine": str: "12:00", // OBBLIGATORIO, FORMATO VALIDO hh:mm, ORAFINE DEVE ESSERE SUCCESSIVA AD ORAINIZIO
"categoria": str: "ambiente", //OBBLIGATORIO, VALORI AMMESSI: "ambiente", "sociale", "educazione", "salute", "altro"
}

→
{
"error": "dati non validi: data deve avere formato dd-mm-yyyy, orafine deve essere successiva ad orainizio, categoria non ammessa"
}

- **500 INTERNAL SERVER ERROR** ⟶ INDICA UN ERRORE INTERNO LATO SERVER. LA RICHIESTA PUÒ ESSERE CORRETTA, MA IL SERVER NON RIESCE A COMPLETARLA. SI USA SE IL SERVER NON RIESCE A SALVARE UNA NUOVA ATTIVITÀ, SE IL DATABASE NON È DISPONIBILE O SE SI VERIFICA UN ERRORE IMPREVISTO DURANTE LA LETTURA DELLE ATTIVITÀ

POST /attività →
← 500 INTERNAL SERVER ERROR

{
"titolo": str: "Pulizia canale", //OBBLIGATORIO, MIN=1, MAX=50 CARATTERI
"descrizione": str: "non lo so", //OBBLIGATORIO, MIN=1, MAX=100 CARATTERI
"luogo": str: "Cagliari", //OBBLIGATORIO
"data": str: "04-06-2026", // OBBLIGATORIO, FORMATO VALIDO dd-mm-yyyy
"orainizio": str: "10:00", // OBBLIGATORIO, FORMATO VALIDO hh:mm
"orafine": str: "12:00", // OBBLIGATORIO, FORMATO VALIDO hh:mm, ORAFINE DEVE ESSERE SUCCESSIVA AD ORAINIZIO
"categoria": str: "ambiente", //OBBLIGATORIO, VALORI AMMESSI: "ambiente", "sociale", "educazione", "salute", "altro"
}

→
{
"error": "errore interno del server. attività non salvata"
}

ESTENSIONE: SOLO ADMIN POSSONO INSERIRE ATTIVITÀ ⟶ PER FARE IN MODO CHE SOLO GLI UTENTI AMMINISTRATORI POSSANO INSERIRE NUOVE ATTIVITÀ, IL SERVER DEVE INTRODURRE UN CONTROLLO DI AUTENTICAZIONE ED AUTORIZZAZIONE. L'UTENTE EFFETUA IL LOGIN ED IL SERVER LO IDENTIFICA TRAMITE UNA SESSIONE O UN COOKIE. AD OGNI RICHIESTA DI CREAZIONE ATTIVITÀ, QUINDI SU POST /attività, IL SERVER RECUPERA L'UTENTE LOGGATO E CONTROLLA IL SUO RUOLO. SE L'UTENTE NON È AUTENTICATO, IL SERVER DEVE RIFIUTARE LA RICHIESTA PERCHÈ NON SA CHI STA TENTANDO L'OPERAZIONE. SE INVECE L'UTENTE È AUTENTICATO MA NON HA RUOLO ADMIN, IL SERVER DEVE RESTITUIRE 403 FORBIDDEN, PERCHÈ L'UTENTE È RICONOSCIUTO MA NON AUTORIZZATO. SOLO SE L'UTENTE È AUTENTICATO ED È AMMINISTRATORE, IL SERVER CREA L'ATTIVITÀ, ASSEGNA IDATTIVITÀ ED ASSEGNA IDORGANIZZATORE A PARTIRE DALL'UTENTE LOGGATO